🌳 PROJET NSI — Terminale

Visualisateur interactif d'arbres binaires et d'ABR

Travail en binôme • Application web • Présentation orale

1. Contexte et objectif du projet

Vous avez étudié les arbres binaires et les arbres binaires de recherche (ABR). Ce projet vous demande de mobiliser ces notions en réalisant une application web interactive qui permet de construire, visualiser et explorer un arbre.

L'objectif n'est pas de produire une interface complexe : c'est de montrer que vous comprenez réellement comment fonctionnent les structures de données arborescentes. Le code Python que vous écrivez est le cœur du projet ; l'interface web n'est qu'un habillage pour le rendre vivant.

🎯 Ce que vous devez démontrer

- Vous savez implémenter un arbre binaire en Python (avec classes ou tuples).
- Vous savez calculer taille et hauteur par récursivité.
- Vous savez réaliser les 3 parcours en profondeur et le parcours en largeur.
- Vous savez construire un ABR, y insérer des éléments et y rechercher une clé.
- Vous savez utiliser l'IA comme outil d'aide — et non comme remplaçant de votre réflexion.

2. Sujet du projet

Vous réaliserez une application web (HTML + JavaScript côté interface, Python côté logique) nommée :

« ArbreViz »

L'application permet à un utilisateur de :

- saisir une liste de valeurs entières,
- choisir un mode : Arbre Binaire générique ou Arbre Binaire de Recherche (ABR),
- visualiser l'arbre construit sous forme graphique dans le navigateur,
- obtenir les résultats des 4 parcours (préfixe, infixe, suffixe, largeur),
- connaître la taille et la hauteur de l'arbre,
- (pour l'ABR) rechercher une clé et voir si elle est présente.

3. Cahier des charges — Fonctionnalités attendues

3.1 Module Python — Logique métier (obligatoire)

Toute la logique algorithmique doit être codée en Python, dans un ou plusieurs fichiers .py.

A. Structure de données

- Implémenter la classe Noeud (ou la version avec 2 classes Noeud + Arbre vue en cours).
- Fournir les fonctions/méthodes : estVide(), gauche(), droit(), get_valeur().

B. Calculs sur l'arbre

- Fonction hauteur(arbre) — récursive, avec la convention racine = niveau 1.
- Fonction taille(arbre) — récursive.

C. Parcours en profondeur (les 3)

- parcours_prefixe(arbre) → renvoie une liste
- parcours_infixe(arbre) → renvoie une liste
- parcours_suffixe(arbre) → renvoie une liste

D. Parcours en largeur

- parcours_largeur(arbre) → renvoie une liste (utiliser une file, comme vu en cours).

E. ABR — fonctionnalités spécifiques

- inserer(abr, valeur) — insertion récursive respectant la propriété ABR.
- rechercher(abr, cle) → renvoie True/False.
- Vérifier que le parcours infixe retourne les valeurs triées.

3.2 Interface web

L'interface doit être fonctionnelle et lisible. Elle n'a pas besoin d'être élaborée graphiquement.

- Un champ de saisie pour entrer les valeurs (ex : « 8, 3, 10, 1, 6 »).
- Un bouton pour générer l'arbre.
- Un affichage visuel de l'arbre (vous pouvez utiliser une bibliothèque JS comme D3.js ou un simple rendu SVG/Canvas — l'IA peut vous aider pour cette partie).
- Un panneau affichant : taille, hauteur, résultats des 4 parcours.
- (ABR uniquement) Un champ pour tester la recherche d'une clé.

⚠ Point d'attention

La communication entre Python et JavaScript doit être expliquée sur la présentation. Vous pouvez utiliser Flask (micro-serveur Python) ou tout autre approche — mais vous devez comprendre et être capables d'expliquer le choix technique que vous avez fait.

4. Usage de l'IA — règles du jeu

L'utilisation de l'IA (ChatGPT, Claude, Copilot…) est autorisée et encouragée pour les parties suivantes :

- Générer le code HTML/CSS/JavaScript de l'interface visuelle.
- Déboguer un message d'erreur que vous ne comprenez pas.
- Générer du code boilerplate (serveur Flask, connexion JS↔Python…).
- Obtenir des explications sur une notion que vous n'avez pas comprise.

⊘ Ce que l'IA ne peut PAS faire à votre place

• Écrire les algorithmes Python du cœur du projet (hauteur, taille, parcours, insertion ABR, recherche).
• Vous devez être capables de ré-expliquer oralement tout code Python présent dans votre projet.
• Si vous ne savez pas expliquer un morceau de code, ne le mettez pas dans le projet.

Lors de la présentation, je vous poserai des questions directement sur votre code Python. Je peux vous demander de modifier une fonction devant moi.

5. Livrables attendus

5.1 Dépôt de fichiers (avant la présentation)

1. Le code source complet de l'application (fichiers .py, .html, .js, éventuellement .css).
2. Un fichier README.md expliquant : comment lancer l'application, les choix techniques, la répartition du travail.
3. Un document (PDF ou .md) nommé journal_ia.md listant les prompts utilisés avec l'IA et ce que vous en avez retenu ou modifié.

5.2 Présentation orale (10 à 15 min par binôme)

La présentation se déroule en 3 phases :

4. Démo live (5 min) — vous montrez l'application qui tourne, vous entrez des valeurs, vous expliquez ce qui se passe.
5. Présentation du code Python (5 min) — vous choisissez 2 fonctions à expliquer en détail (algorithme, récursivité, cas de base).
6. Questions (3–5 min) — je pose des questions sur le code, les choix, l'usage de l'IA.

Conseils pour la présentation

• Préparez-vous à expliquer pourquoi le parcours infixe d'un ABR donne les valeurs triées.
• Préparez-vous à tracer à la main l'arbre généré par une série de valeurs.
• Soyez prêts à expliquer la différence entre un AB et un ABR.
• Si vous avez eu du mal avec une notion, dites-le honnêtement — c'est valorisé.

6. Grille d'évaluation — /20

Critère	Détail	Points
Code Python — Structure	Classes Noeud/Arbre correctement implémentées, interface respectée	2
Code Python — Récursivité	hauteur() et taille() correctes et bien comprises	3
Code Python — Parcours ×3	Préfixe, infixe, suffixe retournant les bonnes listes	3
Code Python — Parcours largeur	Utilisation correcte d'une file, résultat attendu	2
Code Python — ABR	Insertion et recherche correctes, infixe → valeurs triées	3
Interface web	Fonctionnelle, lisible, arbre affiché visuellement	2
Présentation orale	Clarté, maîtrise du code expliqué, réponses aux questions	3
Usage de l'IA	Journal IA fourni, usage réfléchi, code IA non présenté comme sien	2

Total : /20

7. Planning suggéré

Semaine	Travail conseillé
Semaine 1	Implémenter le module Python complet (structure, hauteur, taille, parcours). Tester en console.
Semaine 2	Implémenter la partie ABR (insertion, recherche). Commencer l'interface web avec l'aide de l'IA.
Semaine 3	Connecter Python et JavaScript. Finaliser l'interface. Rédiger README et journal IA.
Semaine 4	Préparer et répéter la présentation orale. Relire et tester le code.

8. Rappels de cours utiles

- 06a_Les_arbres.md — Définitions, TAD, implémentations (tuple, POO v1, POO v2), parcours.
- 06b_Les_ABR.md — Définition ABR, insertion, recherche, parcours infixe trié.
- Activités Capytale : arbre_binaire_tuple, arbre_binaire_POO_v1, arbre_binaire_POO_v2.

Formules à connaître

- Taille d'un arbre complet de hauteur h (racine = niveau 1) : $n = 2^h - 1$
- Encadrement : $\lceil \log_2(n+1) \rceil \leq h \leq n$
- Fils gauche du nœud i (méthode Eytzinger, indice 0) : $2i+1$ | Fils droit : $2i+2$
- Propriété ABR : sous-arbre gauche < racine < sous-arbre droit (pour tout nœud)